# 第1讲
# 编译原理和技术导论

李 诚

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2026年08月31日

# 本节内容提纲

1. 编译器是什么?
2. 为什么要学习编译课程?
3. 编译教学的困境与科大方案
4. AI算子编译比赛分享
5. 本学期课程设置的情况

# 什么是/为什么需要编译器?






- 高级语言
  - 直接面向开发者
  - 与数学公式类似
  - 编程效率高
- 机器语言
  - 驱动硬件完成具体任务
  - 编程效率低
- 编译器提供程序开发的便捷性
  - 实现人机交流，将人类易懂的高级语言翻译成硬件可执行的目标机器语言

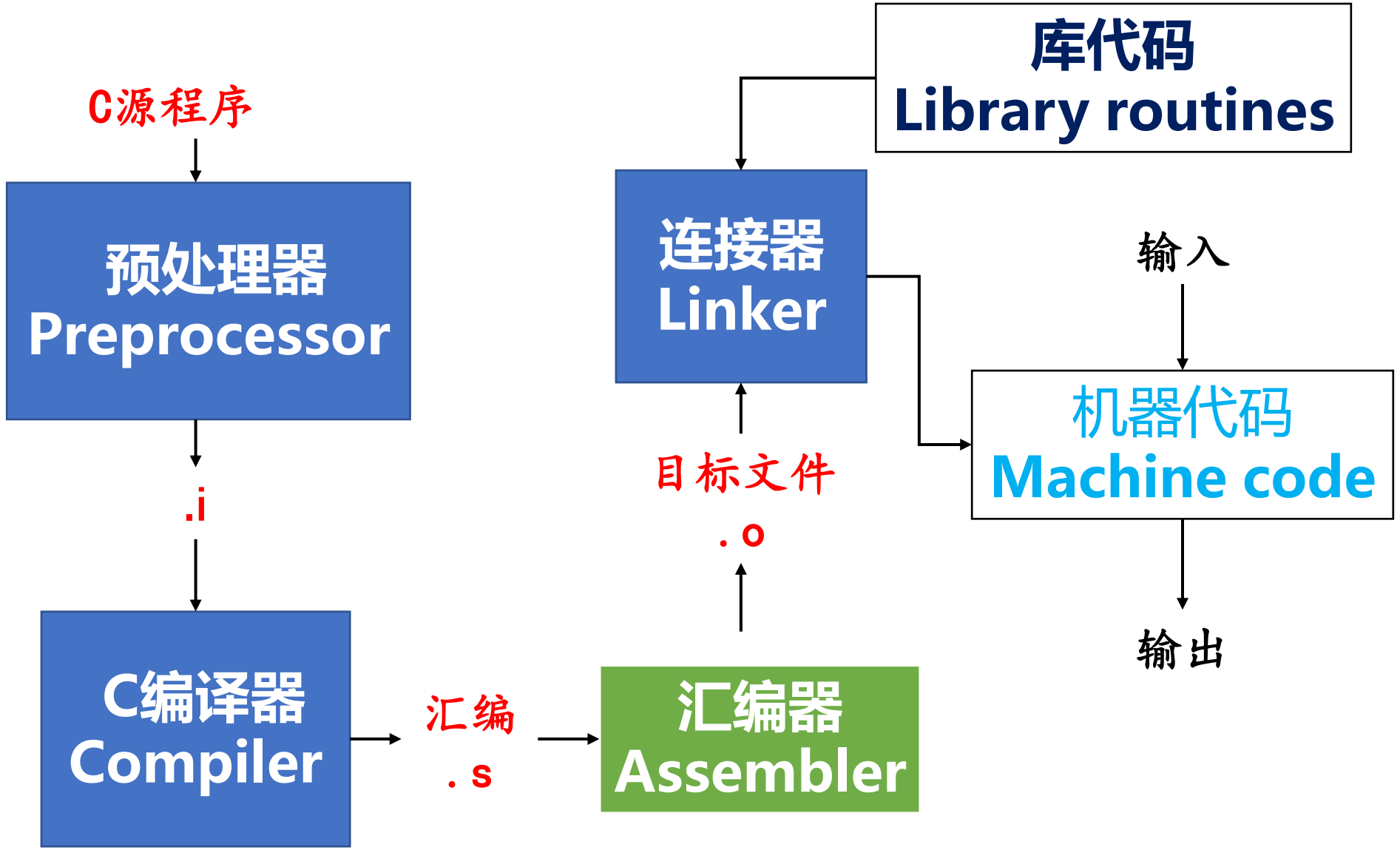
# 从C程序到可执行文件

```c
#include <stdio.h>
int main()
{
  printf("hello, world!\n");
}

/* helloworld.c */
```

```
[root@host ~]# clang helloworld.c -o helloworld
[root@host ~]# ./helloworld
hello, world!
```

# C程序的编译运行过程分解




参考链接：http://www.cs-fundamentals.com/c-programming/how-to-compile-c-program-using-gcc.php

# 编译器自动优化能力







| 优化等级 | 简要说明 |
| --- | --- |
| -Ofast | 在-O3级别的基础上，开启更多**激进优化项**，该优化等级不会严格遵循语言标准 |
| -O3 | 在-O2级别的基础上，开启了更多的**高级优化项**，以编译时间、代码大小、内存为代价获取更高的性能。 |
| -Os | 在-O2级别的基础上，开启**降低生成代码体量**的优化 |
| -O2 | 开启了大多数**中级优化**，会改善编译时间开销和最终生成代码性能 |
| -O/-O1 | 优化效果介于-O0和-O2之间 |
| -O0 | 默认优化等级，即**不开启编译优化**，只尝试减少编译时间 |

延伸阅读：https://clang.llvm.org/docs/CommandGuide/clang.html#code-generation-options

# 举例——优化对代码性能的影响

## ❑1000000000次循环迭代累加

```
#include <stdio.h>
#include <time.h>
int main() {
    int loop = 1000000000;
    long sum = 0;
    int start_time = clock();
    int index = 0;
    for (index = 0; index < loop; index ++)
    {
        sum += index;
    }
    int end_time = clock();
    printf("Sum : %ld, Time Cost : %lf \n", sum, (end_time - start_time) * 1.0 / CLOCKS_PER_SEC);
    return 0;
}
```

循环次数定义

开始计时

循环体

结束计时

代码运行时间输出

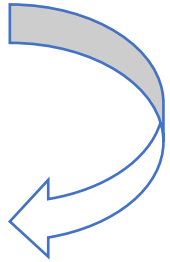
# 举例——优化对代码性能的影响

- gcc –O0 无优化执行

```
 gloit@gloit-x1c  ~/2022_compiler_demo  master  gcc -O0 add.c
 gloit@gloit-x1c  ~/2022_compiler_demo  master  ./a.out
Sum: 49999999500000000, Time Cost: 3.415244
```

- gcc –O1 中级优化执行

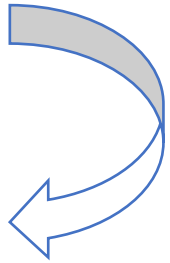
```
 gloit@gloit-x1c  ~/2022_compiler_demo  master  gcc -O1 add.c
 gloit@gloit-x1c  ~/2022_compiler_demo  master  ./a.out
Sum: 49999999500000000, Time Cost: 0.554717
```



性能提升5倍

- gcc –O2 高级优化执行

```
 gloit@gloit-x1c  ~/2022_compiler_demo  master  gcc -O2 add.c
 gloit@gloit-x1c  ~/2022_compiler_demo  master  ./a.out
Sum: 49999999500000000, Time Cost: 0.000002
```



性能提升数十万倍

# 编译器的输入




- 标准的指令式语言(Java, C, C++)
  - 状态
    - 变量
    - 结构
    - 数组
  - 计算
    - 表达式 (arithmetic, logical, etc.)
    - 赋值语句
    - 条件语句 (conditionals, loops)
    - 函数

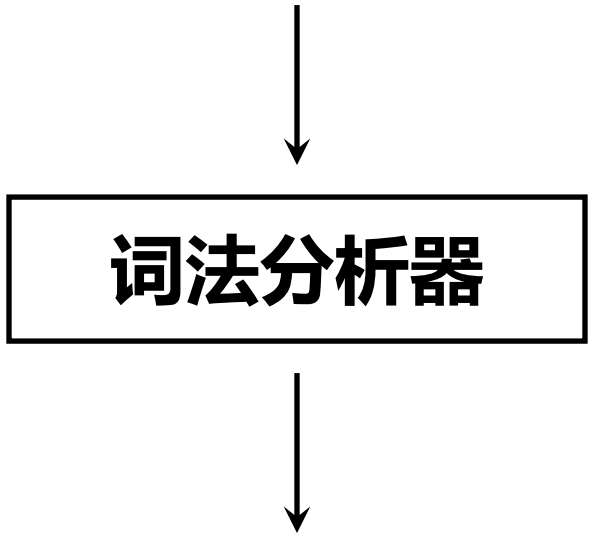
# 编译器的输出

- 状态
  - 寄存器
  - 内存单元
- 机器码 – load/store architecture
  - Load, store instructions
  - 寄存器操作– Arithmetic, logical operations
  - 分支指令– Branch instructions

# 词法分析

- 将程序字符流分解为记号（Token）序列
  - 形式：<token_name, attribute_value>

position = initial + rate * 60 ← 字符流




符 号 表

| | | |
|---|---|---|
| 1 | position | · · · |
| 2 | initial | · · · |
| 3 | rate | · · · |
| | | |

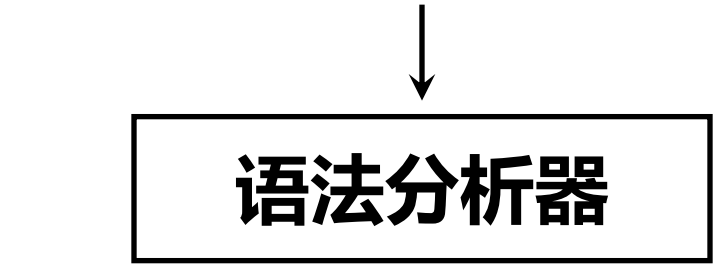
⟨id, 1⟩ ⟨=⟩ ⟨id, 2⟩ ⟨+⟩ ⟨id, 3⟩ ⟨*⟩ ⟨60⟩ ← 记号流

命令行输入：clang -cc1 -dump-tokens xx.c

# 语法分析

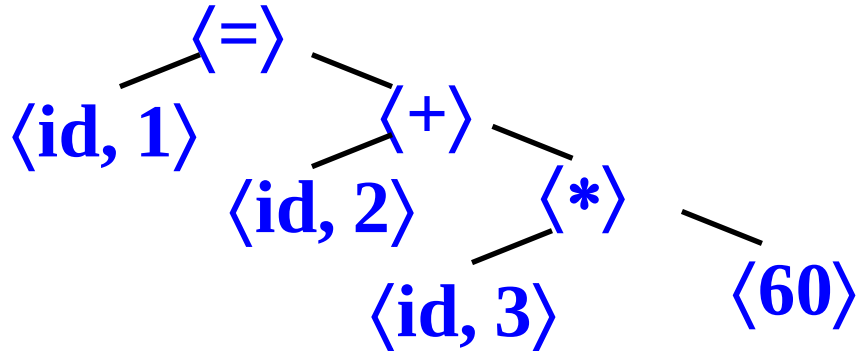
- 也称为解析（Parsing），在词法记号的基础上，创建语法结构

⟨id, 1⟩ ⟨=⟩ ⟨id, 2⟩ ⟨+⟩ ⟨id, 3⟩ ⟨*⟩ ⟨60⟩ ← 记号流




符 号 表

| | | |
|---|---|---|
| 1 | position | · · · |
| 2 | initial | · · · |
| 3 | rate | · · · |
| | | |




← 语法树

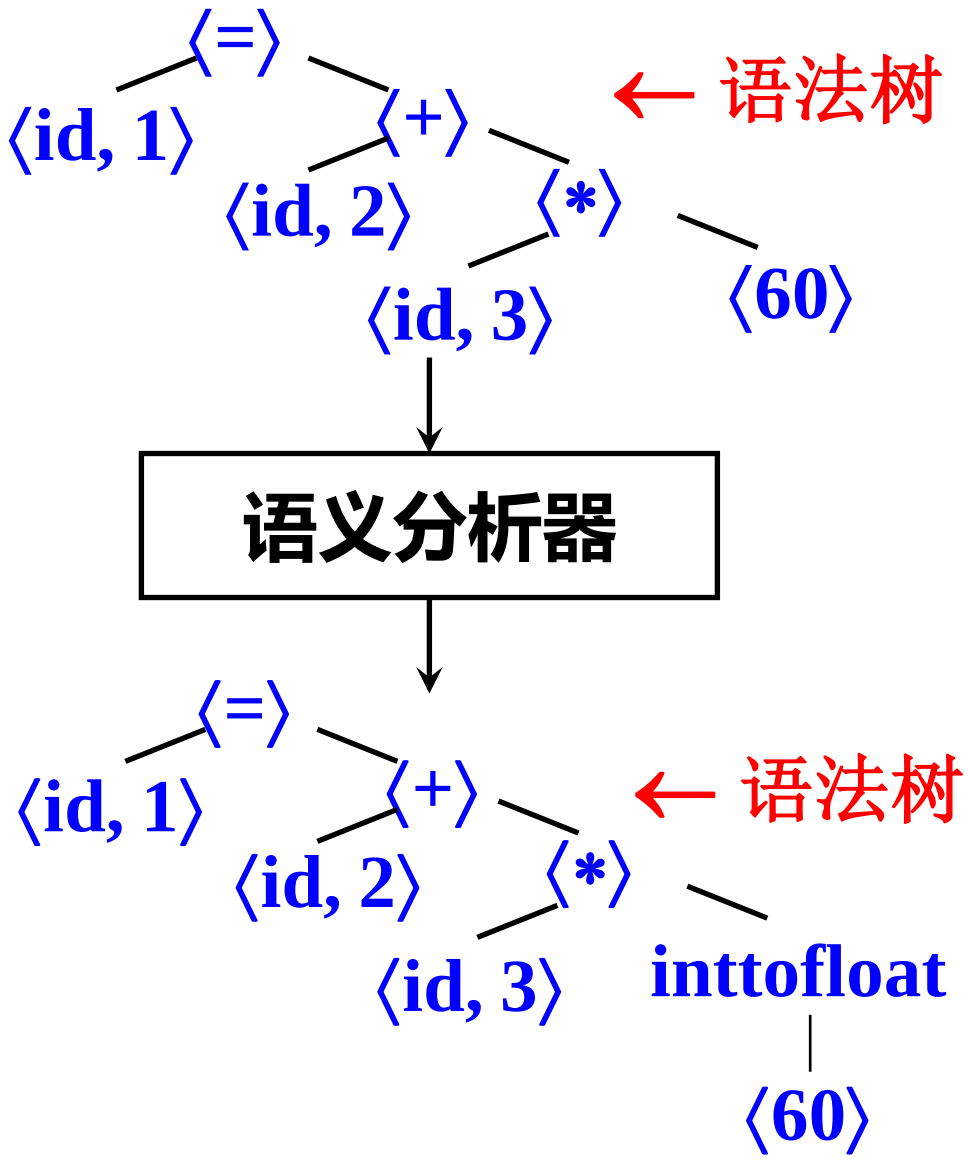
命令行输入：clang -fsyntax-only -Xclang -ast-dump xx.c

# 语义分析

- 编译器会检查程序中的不一致
  - 如：类型检查（type checking）

符 号 表

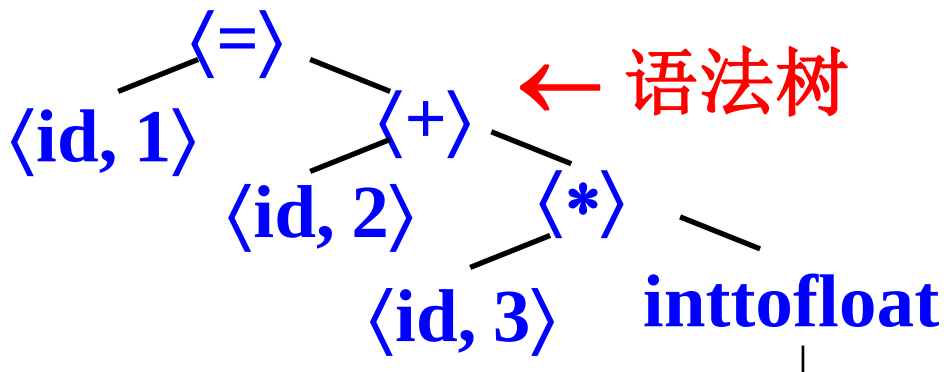
| | | |
|---|---|---|
| 1 | position | · · · |
| 2 | initial | · · · |
| 3 | rate | · · · |
| | | |




注：类型转换在astdump时已经完成

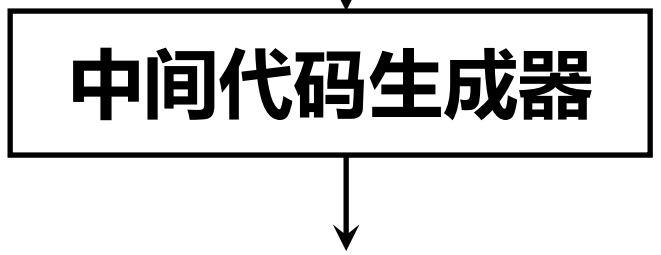
# 中间代码生成

- 是源语言与目标语言之间的桥梁




符 号 表

| | | |
|---|---|---|
| 1 | position | · · · |
| 2 | initial | · · · |
| 3 | rate | · · · |
| | | |




```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

← 中间代码

命令行输入：clang -cc1 xx.c -emit-llvm -o xx.ll

# 机器无关代码优化

- 机器无关的代码优化便于生成执行时间更快、更短或能耗更低的目标代码

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

← 中间代码

符 号 表

| | | |
|---|---|---|
| 1 | position | · · · |
| 2 | initial | · · · |
| 3 | rate | · · · |
| | | |

↓

**代码优化器**

↓

```
t1 = id3 * 60.0
id1 = id2 + t1
```

← 中间代码

命令行输入：clang -S -emit-llvm -O3 test.c -o test-new.ll

# 目标代码生成

- 如果目标语言是机器代码，必须为变量选择寄存器或内存位置

```
t1 = id3 * 60.0
id1 = id2 + t1
```
← 中间代码

↓

**代码生成器**

↓

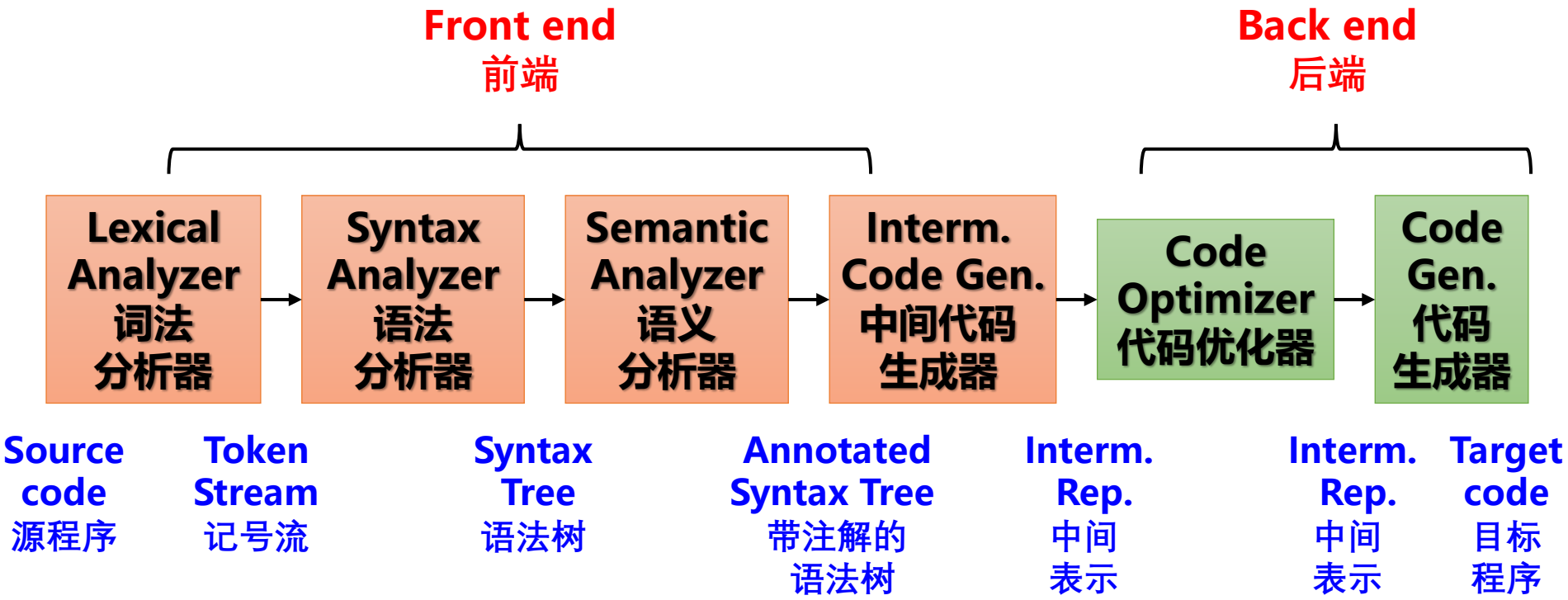
符 号 表

| | | |
|---|---|---|
| 1 | position | . . . |
| 2 | initial | . . . |
| 3 | rate | . . . |
| | | |

```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```
← 汇编代码

命令行输入：llc-14 xxx.ll -o xxx.s

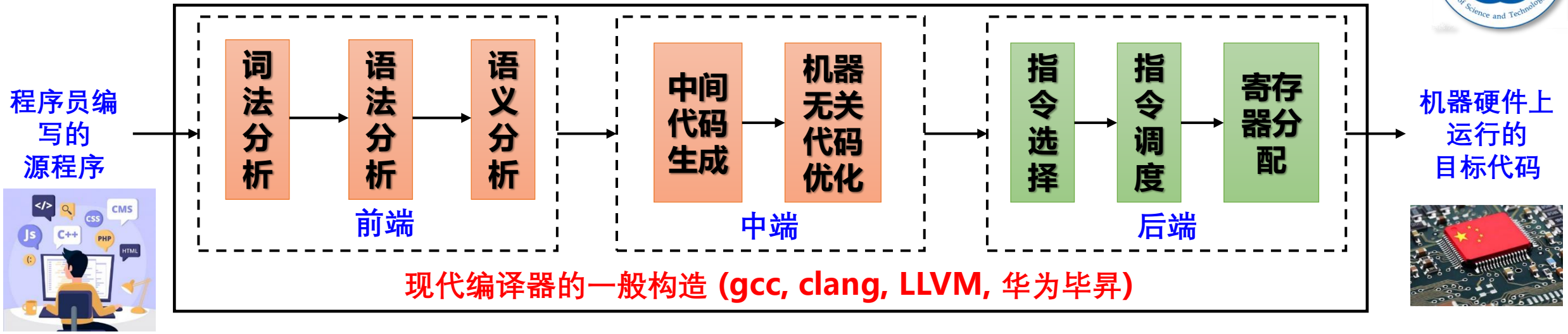
# 现代编译器的构造/阶段


Front end
前端
Back end
后端
Lexical Analyzer 词法分析器
Syntax Analyzer 语法分析器
Semantic Analyzer 语义分析器
Interm. Code Gen. 中间代码生成器
Code Optimizer 代码优化器
Code Gen. 代码生成器
Source code 源程序
Token Stream 记号流
Syntax Tree 语法树
Annotated Syntax Tree 带注解的语法树
Interm. Rep. 中间表示
Interm. Rep. 中间表示
Target code 目标程序
Symbol Table 符号表
Error Handler 错误处理

# 本节内容提纲

1. 编译器是什么?
2. 为什么要学习编译课程?
3. 编译教学的困境与科大方案
4. AI算子编译比赛分享
5. 本学期课程设置的情况

# 编译课程简介




“编写编译器的原理和技术具有普遍的意义，以至于在每个计算机科学家的研究生涯中，该书中的原理和技术都会反复用到。”

——著名计算机专家 *Alfred V. Aho*

“在供应链可控性上，仍存在编译工具依赖国外的情况。”

“我们应重点突破操作系统内核、编译器等关键技术。”

——《中国信息技术产品安全可控年度发展报告》

# 编译技术历史悠久影响深远

- 图灵奖自1966年颁发以来，共有76名获奖者，其中编译相关的科研人员有21位，占比27.6%，Alan J. Perlis因编译技术贡献成为第一位获得图灵奖的科学家。

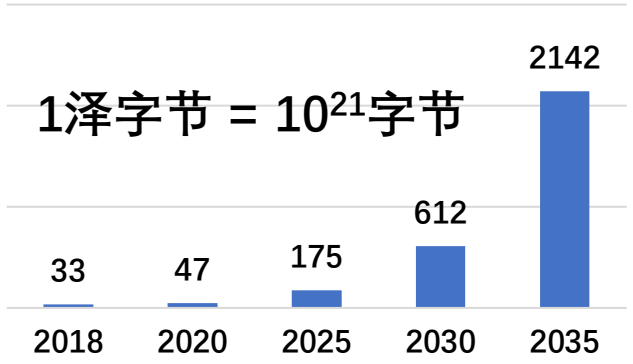
| 年份 | 科学家 | 贡献 | 年份 | 科学家 | 贡献 |
|---|---|---|---|---|---|
| 1966 | Alan J. Perlis | 高级程序设计技巧，编译器构造 | 1972 | Edsger Dijkstra | 程序设计语言的科学与艺术 |
| 1974 | Donald E. Knuth | 算法分析、程序设计语言的设计、程序设计 | 1976 | Michael O. Rabin<br>Dana S. Scott | 非确定性自动机 |
| 1977 | John Backus | 高级编程系统，程序设计语言规范的形式化定义 | 1979 | Kenneth E. Iverson | 程设语言和数学符号，互动系统的设计，程设语言的理论与实践 |
| 1980 | C. Antony R. Hoare | 程序设计语言的定义与设计 | 1983 | Ken Thompson<br>Dennis M. Ritchie | UNIX操作系统和C语言 |
| 1984 | Niklaus Wirth | 程序设计语言设计、程序设计 | 1987 | John Cocke | 编译理论，大型系统的体系结构，及RISC计算机的开发 |

# 编译技术历史悠久影响深远

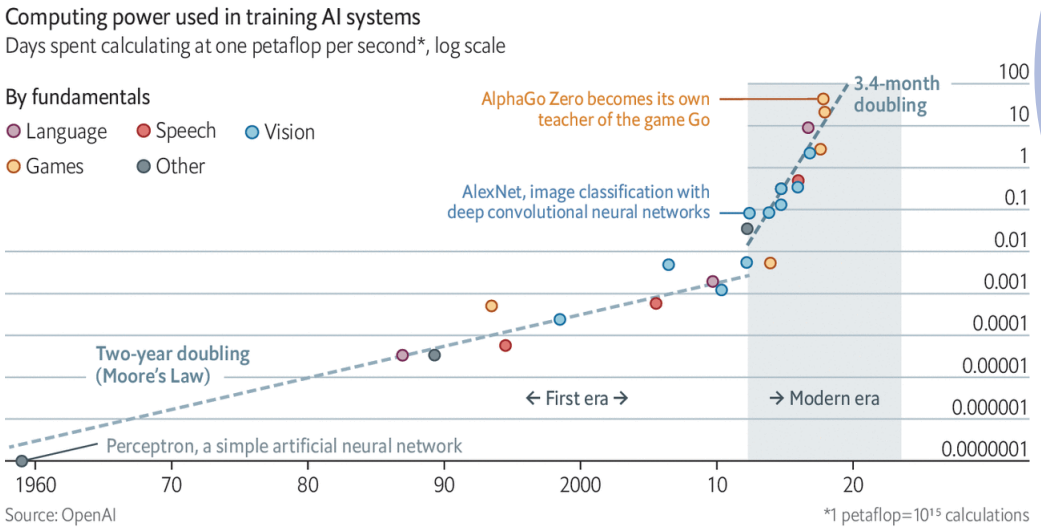
- 图灵奖自1966年颁发以来，共有76名获奖者，其中编译相关的科研人员有21位，占比27.6%，Alan J. Perlis因编译技术贡献成为第一位获得图灵奖的科学家。

| 年份 | 科学家 | 贡献 | 年份 | 科学家 | 贡献 |
|---|---|---|---|---|---|
| 2001 | Ole-Johan Dahl Kristen Nygaard | 面向对象编程 | 2003 | Alan Kay | 面向对象编程 |
| 2005 | Peter Naur | Algol 60语言 | 2006 | **Frances E. Allen** | 优化编译器 |
| 2008 | **Barbara Liskov** | 编程语言和系统设计的实践与理论 | | | |
| 2020 | **Jeffrey David Ullman** | **Alfred Vaino Aho** | **创造了全球数百万编程人员使用的工具和教材，推进编程语言实现的基础算法和理论，并在极具影响力的书籍中综述了这些研究成果** | | |
| 2021 | **Jack J. Dongarra** | 通过对线性代数运算的高效数值算法、并行计算编程机制和性能评估工具的贡献，引领了高性能计算的世界。 | | | |

# 智能时代的开启

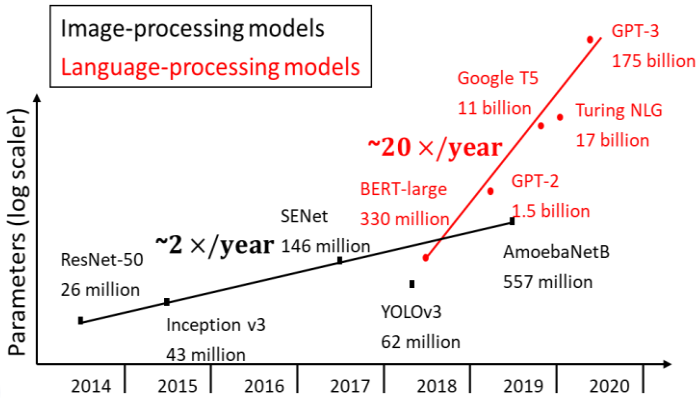



数字经济、数字孪生助推
全社会数据的爆炸式增长



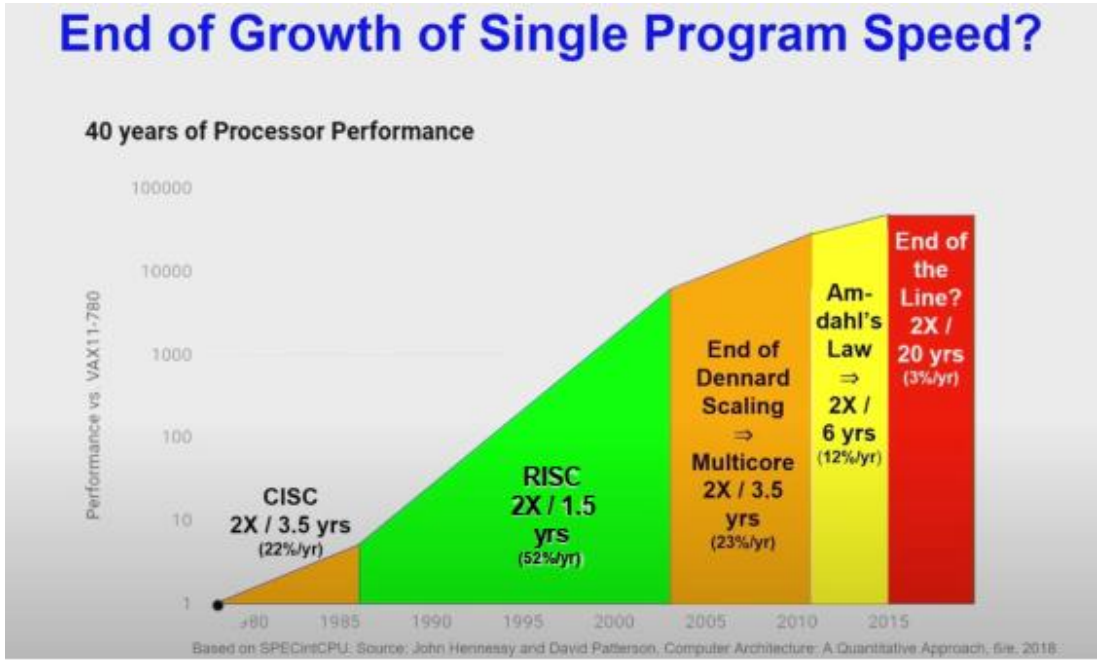

过去十年，AI训练算力需求增
长了30万倍，3.4个月翻一倍

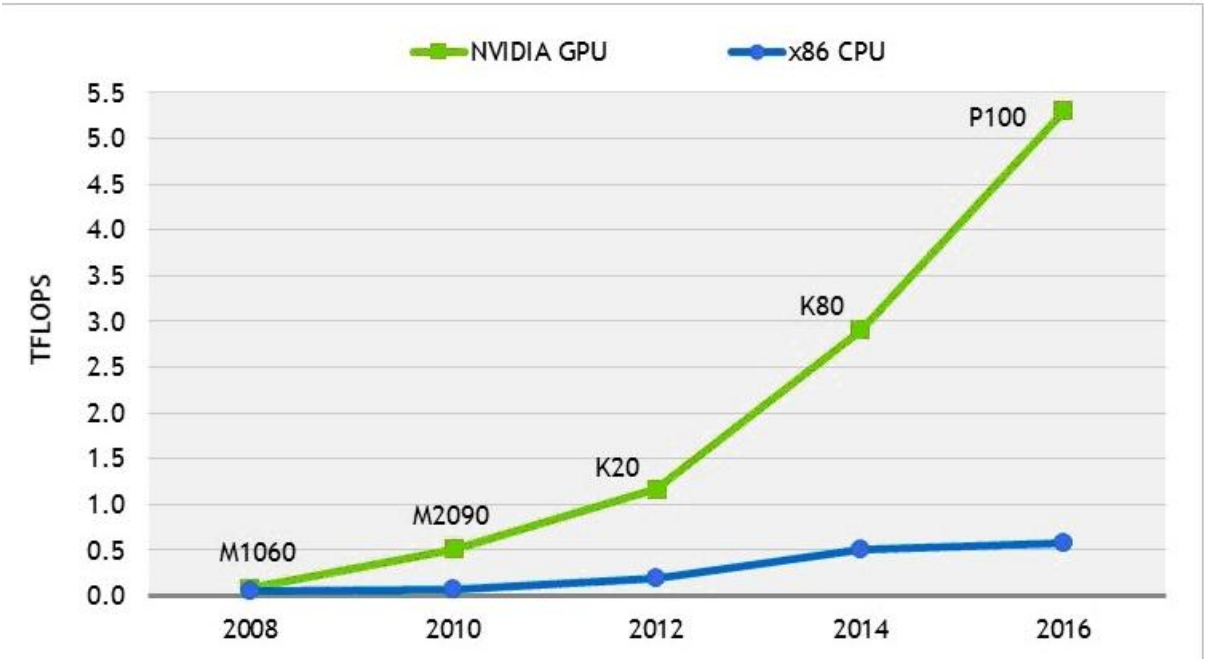



新版摩尔定律：
宇宙中的智能数量每18个
月就会翻一倍

# 体系结构的黄金时代



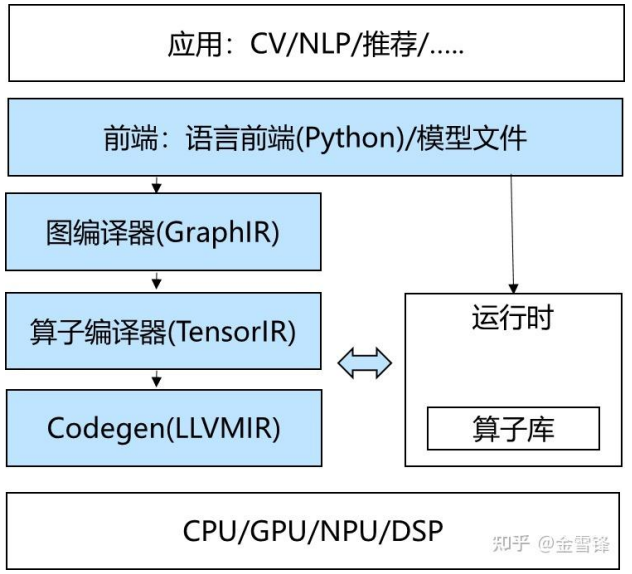

Turing Lecture, Hennessy, Patterson; June 2018 / CACM Feb 2019

https://www.rtinsights.com/gpus-the-key-to-cognitive-computing/

- 领域专用芯片的使用助推了人工智能的发展

# 编译是工业界关注的焦点

- A New Golden Age for Computer Architecture
  - By John L. Hennessy, David A. Patterson；Communications of the ACM
- 新硬件推陈出新的时代
  - GPU、DPU、TPU、NPU、xPU
  - 量子计算机
- 新应用不断涌现的时代
  - AI计算、科学计算（AlphaFold）、量子计算
- 编译技术的需求日益旺盛
  - 华为方舟、毕昇编译器、龙芯二进制翻译、AI编译器（XLA、TVM）



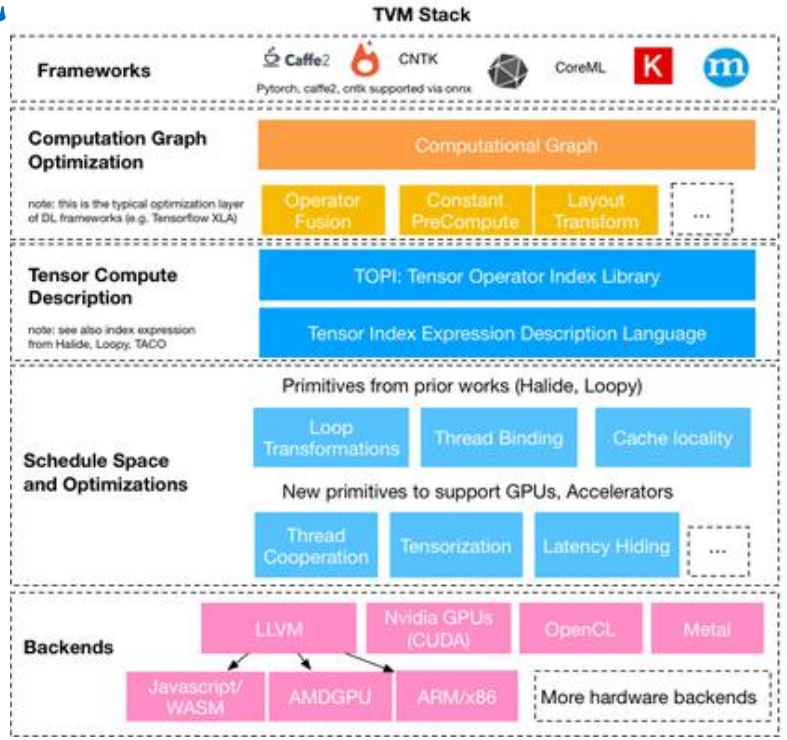
# 体系结构黄金时代的机会

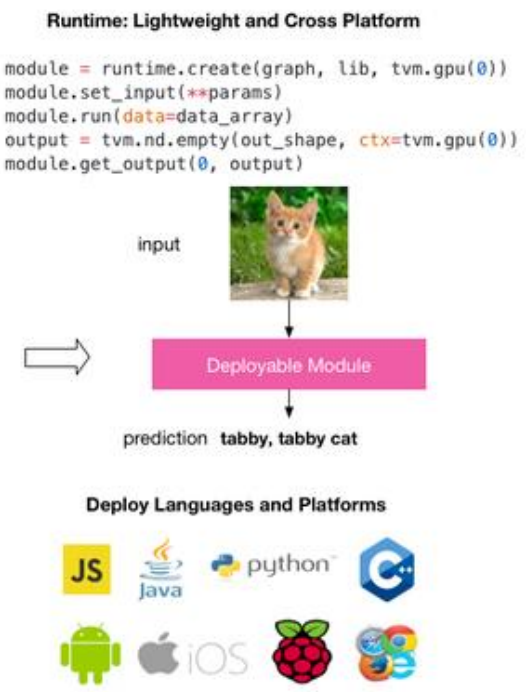
- 软件为中心
  - 现代编程语言很多是script语言-> 需要解析、动态类型、代码复用
  - 编程高效但是执行不见得高效：Python vs. C
- 硬件为中心
  - 唯一可行的路径是领域专用架构
  - 一颗芯片只做一部分事情，但是性能很好
- 软硬协同的设计思路
  - 专用领域语言+专用领域架构：CUDA + GPU
  - 可以最大化挖掘硬件的并行度

# 新一代编译技术呼之欲出

## ❑新一代编译器需要具备以下基础能力

- 为异构芯片提供统一的硬件抽象
- 支持为异构芯片生成对应的目标代码
- 能理解专用领域的语言和编程范式
- 编译框架自身的质量、可靠性和性能



# 产业界编译人才紧缺—字节



字节跳动招聘 ByteDance Recruitment 校招

首页 技术人才项目 职位 招聘动态 产品与技术

## AI芯片工具链软件工程师 - 芯片研发

北京、深圳、上海、杭州

正式 | 研发 - 硬件 | 2027届校园招聘 | 职位 ID：A211575A

### 职位描述

团队介绍：字节芯片研发团队目前工作主要集中在芯片设计环节，主要围绕字节自身业务展开芯片探索，为字节多项业务的专用场景定制硬件优化，设计多款基于先进半导体工艺的云端复杂芯片，以期提升性能、降低成本。早期若干芯片项目已经进入到量产部署阶段，有多次一版成功的投片经历，所用工艺包含多个主流的先进工艺节点。与基础设施各团队之间工作的整体协同，能更容易和更好地发挥芯片研发的价值。

1、参与整体AI工具链的研发过程，包括以下主要工作内容；
2、参与AI编译器的开发，模型量化等特性支持；
3、参与算子开发、算子编译及调优，模型编译支持及优化；
4、参与Runtime框架研发；
5、软硬件协同优化、AI加速器评测相关工作；
6、业界前沿模型调研工作。

## 职位要求

1、2027届获得本科及以上学历，专业不限；
2、**理解深度学习框架和编译软件栈；熟悉C++、Python等编程语言；掌握数据结构与算法、计算机体系结构等基础知识；有主动学习、快速解决问题的能力和自我驱动力。**

加分项：

**1、了解编译器and代码优化技术，熟悉MLIR、TVM、XLA、LLVM等中间表示，有相关开发经验；**
**2、对常用的深度学习框架有深入理解，包括但不限于TensorFlow、PyTorch、MXNet；**
**3、有AI加速硬件的量化工具开发经验；**
**4、有GPU、FPGA或AI芯片相关的开发和评测经验；**
**5、有LLM算子开发和部署经验。**

# 产业界编译人才紧缺—华为

编译器 [杭州·滨江区·长河]

25-50K 经验不限 本科 刘女士 | 编译器与芯片...

华为

计算机软件 不需要融资 10000人以上

Go | 后端开发

加班补助，补充医疗保险，定期体检，五险一金，年终奖，...

编译器开发 [西安·雁塔区·高新软件园]

20-40K·16薪 经验不限 硕士 陈先生 | 研发工程师

龙芯中科

电子/半导体/集成电路 已上市 500-999人

编译器 | Loongarch | 龙芯

定期体检，住房补贴，生日福利，员工旅游，包吃，补充医...

编译器工程师 [上海·浦东新区·金桥]

18-35K 1-3年 本科 王先生 | 软件研发工程师

华为技术有限公司

计算机软件 不需要融资 10000人以上

数据结构 | 架构师 | ARM开发

住房补贴，定期体检，离职补偿，零食下午茶，补充医疗保...

编译器工程师 [上海·徐汇区·徐家汇]

20-30K·15薪 经验不限 本科 韩女士 | 技术经理

麒麟软件

计算机软件 未融资 1000-9999人

算法基础 | Linux | 编译工具链 | 汇编 | 反汇编

年终奖，免费班车，员工旅游，定期体检，五险一金，员工...

## 职位信息

职位描述：

1、负责基于编译技术的字节跳动核心业务的性能分析及优化；

2、负责各种编程语言(C++/Java/Go/Rust/WebAssembly)及周边基础库在业务中优化、技术推广与生态建设；

3、负责AutoFDO相关优化落地的研发维护；

4、负责前沿编译优化技术在字节跳动核心业务的落地；

5、负责编译技术针对X86_64, ARM64, RISC-V等体系结构的功能开发与性能优化；

6、负责异构编译技术针对AI领域与异构硬件的功能开发与性能优化；

7、参与研发业界领先的性能分析及优化平台。

职位要求：

1、熟悉编译原理以及相关编译优化技术；

2、熟悉LLVM/GCC/GraalVM/OpenJDK/Go中某个编译器框架或者Runtime；

3、熟悉C++/Java/Go/Rust/WebAssembly中某个语言的设计与实现原理；

4、熟悉Intel/AMD x86_64或ARM64、RISC-V体系结构，精通x86_64、ARM64、RISC-V微架构；

5、精通C/C++，熟悉C++底层实现原理；

6、有以下经验者优先：

a. 熟悉LLVM/GCC等C/C++编译器，并参与相关社区者优先；

b. 有HPC/AI编译器以及高性能库开发经验者优先。

职能类别：系统架构设计师

# 本节内容提纲

1. 编译器是什么?
2. 为什么要学习编译课程?
3. 编译教学的困境与科大方案
4. AI算子编译比赛分享
5. 本学期课程设置的情况

# 编译教学及人才培养目标




计算机专业核心课

**编译原理和技术**

编译器

- ★ 一般构造原理
- ★ 基本设计方法
- ★ 主要实现技术

有助于培养学生的：

- **计算机系统能力**
- **综合、创新能力**
- **软件工程能力**

目标1：从事编译相关产业的领军人才

目标2：从事编译相关产业的技术骨干

目标3：使用编译技术解决其他关键问题的人才

# 智能时代编译人才培养困境

## 《人民日报》：大模型的竞争，是国家科技战略的竞争

点击：4953 作者：记者 来源：战略前沿技术微信号 发布时间:2023-09-23 09:32:58



今年以来,ChatGPT引领了人工智能的新一轮创新浪潮。科技部新一代人工智能发展研究中心发布的《中国人工智能大模型地图研究报告》显示,我国研发的大模型数量位居全球第二。据不完全统计,当前中国已发布的大模型产品已超过80个,进入“百模大战”的新时代。这充分体现了我国在人工智能领域的创新实力和发展潜力,同时如何走出具有中国特色的大模型发展之路,也成为我们必须思考和回答的重大课题。

## 自主创新与核心竞争力构建的挑战

① 算力被国外封锁

② 训练框架依赖国外

③ 顶尖系统人才缺乏

提出

回应

## 立德树人的国家要求与时代需求

**围绕自主可控芯片技术**
**培养兼具系统能力和创新精神的**
**复合型高性能计算机系统人才**

数据来源：人民日报，2023.7.20，郑纬民院士文章

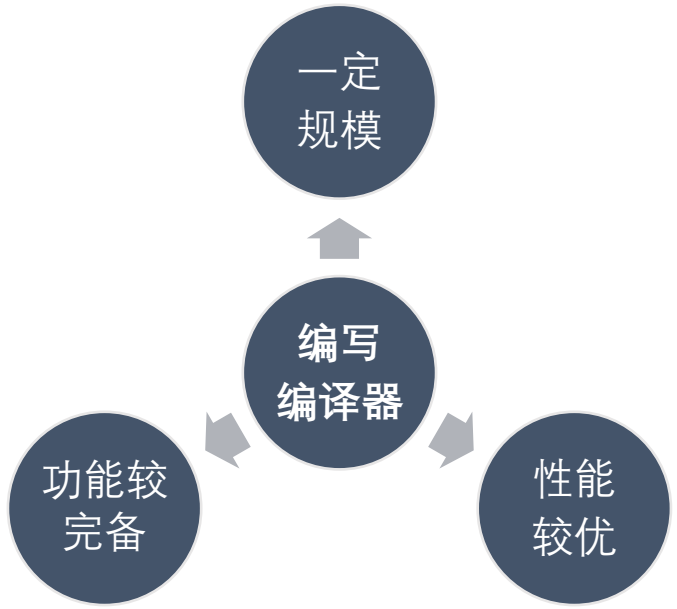
# 国内外实践体系调研

| | | 前端语言类型 | 词法器 | 语法器 | 中间代码生成 | 后端代码生成 | 中间代码优化 | 寄存器分配 |
|---|---|---|---|---|---|---|---|---|
| 国外高校 | ETH | | √ | √ | LLVM | x86 | √ | √ |
| | 剑桥 | 基于Ocaml | √ | √ | Ocaml IL | ARM、RISC-V | √ | √ |
| 国内高校 | 清华大学(交叉院) | C/C++/Python | √ | √ | √ | RISC-V | × | √ |
| | 哈工大 | | √ | √ | √ | × | × | × |
| | 东北大学 | PASCAL | √ | √ | 四地址码 | x86 | × | × |
| | 东北师大 | × | × | × | × | × | × | × |
| | 中科大 | C/C++ | √ | √ | LLVM | LoongArch | √ | √ |

- 部分高校未开展编译实践；部分学校未贯通编译实践的全部环节；LLVM重要但未被广泛采用；x86和RISC-V仍是主流；国产自主体系未体现

# 编译实验体系总览

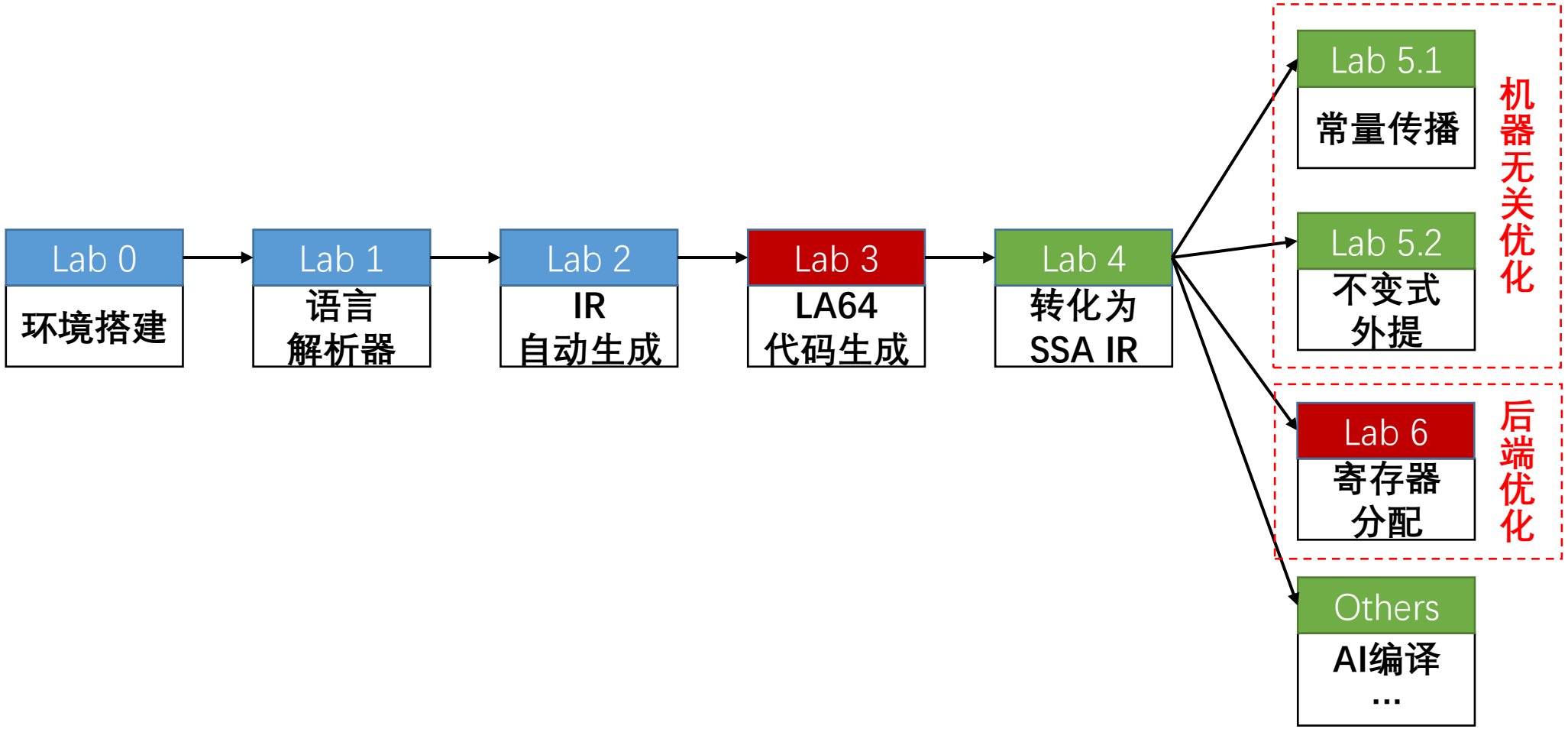
- **目标：面向国家战略需求，以系统和创新能力的培养为导向，根据计算机学科发展的趋势和新时代大学生的成长需要构建现代编译实验体系，体现先进性、挑战度，增强学生获得感**




| | |
|---|---|
| 源语言特性 | 基于Cminus语言，增加对浮点数、数组等复杂特性的支持 |
| 词法语法分析器 | 使用Flex+Bison工具构造编译器前端，重点考察对Cminus语言正则表达式和语法产生式的抽象能力 |
| 中间代码生成器 | 使用简化版LLVM中间表达子模块和基于访问者模式的编程框架(自研)，自动生成 Cminus程序的中间代码 |
| 中间代码优化器 | 使用封装好的基本块、调用者使用者链表等常用数据结构及其接口(自研)，实现简单/复杂代码优化算法 |
| 目标代码生成器 | 支持龙芯(与龙芯合作研发)、RISC-V等多指令集后端，鼓励对寄存器分配算法、指令选择算法等进行探索验证 |

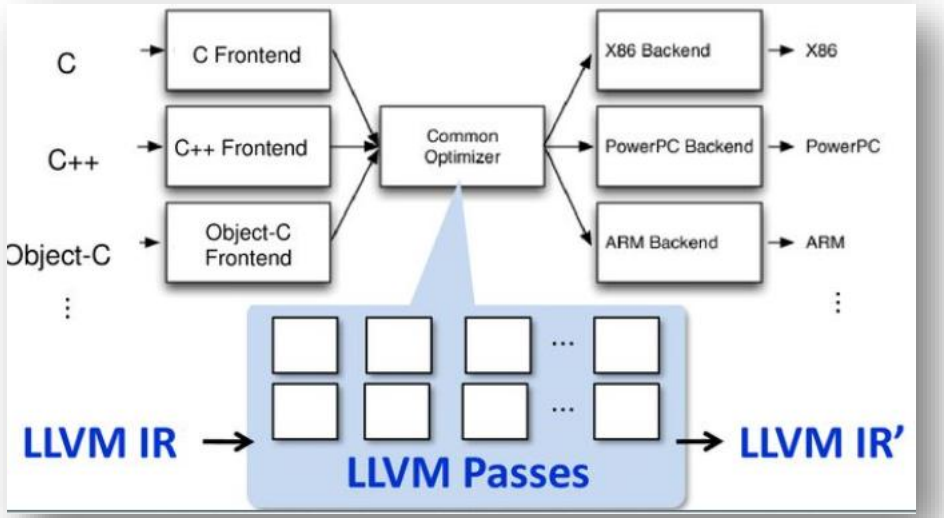
# 编译实验架构图

- 基础实验+龙芯后端实验+高阶创新实验有机结合

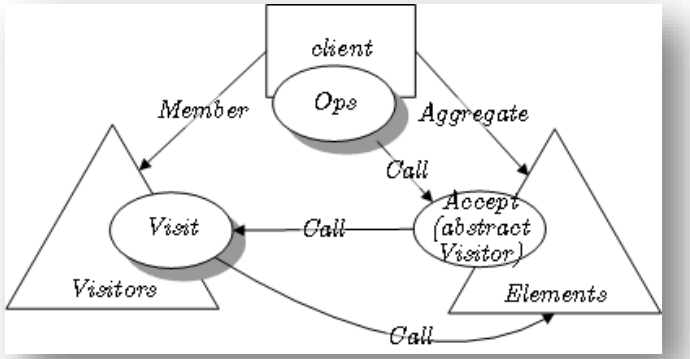


# 编译实验的改革——先进性

- 紧跟时代前沿，融合先进技术，架起课程教育与工业界技术革新之间的桥梁




在全国范围内较早地将**LLVM——这一工业界广泛使用的、产品级现代编译工具链**引入到课程实验中，建立贴近业界需求、适合多种教学目标的循序渐进的课程实践体系




使用适合编译器编写的**访问者设计模式**，将数据结构与操作分离开来，便于接口扩展，降低编程门槛




支持**LoongArch后端**，为自主可控芯片产业培养专业人才

# 编译教材的撰写与数字资源建设

## 图 书 出 版 合 同

高出字第 20241128 号

**甲方（著作权人）：李诚、徐伟 、张昱**

代 理 人：李诚

身份证号：522101198706073618

单　　位：中国科学技术大学

通信地址：安徽 合肥 蜀山区中国科大西区计算机学院

邮政编码：230000

联系电话：18956001767

**乙方（出版者）：高等教育出版社有限公司**

住　　所：北京市西城区德胜门外大街4号

通信地址：北京市朝阳区惠新东街4号富盛大厦

邮政编码：100029

联 系 人：王康

部　　门：工科事业部计算机分社

电　　话：010-58556021

根据《中华人民共和国著作权法》《中华人民共和国著作权法实施条例》《中华人民共和国民法典》等法律、法规及有关规定，甲乙双方就作品出版达成如下协议。

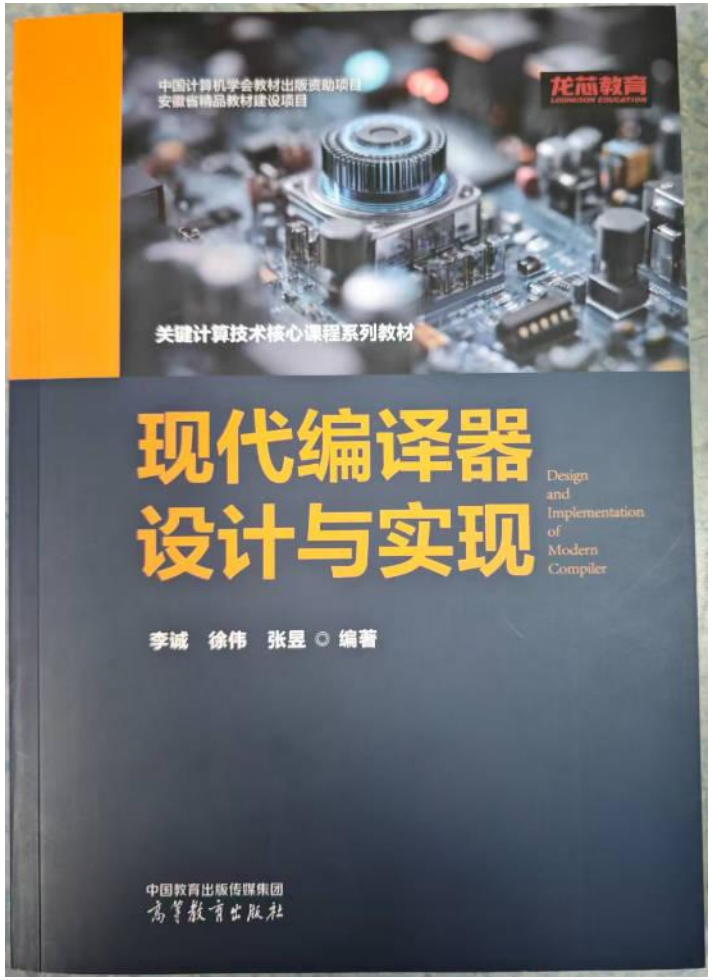
**第一条** 作品名称：现代编译器设计与实现（以下简称“本作品”）。

本作品字数约 500 千字，插图约 150 幅。

主要读者对象：计算机类本科生、研究生 。

甲方将本作品用中文简体、中文繁体，以图书、音像制品、电子出版物、网络出版物、新媒体（除纸质出版物、音像制品、电子出版物、网络出版物之外的，目前已知和今后产生的新的出版形式）的单一或者组合载体形式，在世界范围内的专有出版权许可给乙方。

**第二条** 甲方保证拥有本作品的著作权并有权许可乙方使用。如果本作品含有侵犯他人著作权、名誉权、肖像权、姓名权等权益的内容，或乙方按本合同约定行使甲方许可的权利而侵犯他人的合法权益致乙方被第三方索赔或起诉，甲方应赔偿由此给乙方造成的损失，乙方有权直接从未支付给甲方的稿酬中抵扣并可解除本合同。




现代编译器设计与实现

李诚、徐伟，现代编译器设计与实现（实验讲义版本，高等教育出版社待出版，2023）

教材代码

下载教材代码

实验框架

- 实验一：语言解析器
- 实验二：IR 自动生成
- 实验三：LA64 代码生成
- 实验四：转化为 SSA IR
- 实验五：机器无关优化
- 实验六：寄存器分配

目录

教材代码
实验框架
编译器设计与实现慕课
龙芯慕课*
其他资源
教材更新与联系人
教材勘误

**电子书：https://ustc-compiler-principles.github.io/textbook**

中共安徽省委教育工作委员会
安 徽 省 教 育 厅

皖教工委函〔2023〕305号

中共安徽省委教育工委 安徽省教育厅关于公布2023年高校“三全育人”综合改革和思想政治

通知

面向自主指令集的编译器设计与实现

| 序号 | | | 申报高校 |
|---|---|---|---|
| 1 | | | |
| 2 | 面向自主指令集的编译器设计与实现 | 李诚 | 中国科学技术大学（省科学教育教材建设重点研究基地） |
| 3 | 拓扑学讲义 | 王作勤 | |

**已由高等教育出版社出版**；入选**全国系统能力大赛组委会专家组推荐教材**；入选**安徽省精品教材**计划（**34项中唯一**计算机核心课）

# 编译教材的撰写与数字资源建设




2024年2月，实验教材讲解视频上传至B站，累计**10.1万人次观看**

# 全国大学生系统能力大赛编译比赛战绩

- 2020-2026，共获奖励20项

| 奖励类型 | 数量 |
| --- | --- |
| 特等奖 | 1 |
| 一等奖 | 3 |
| 二等奖 | 5 |
| 三等奖 | 7 |
| 优胜奖 | 4 |



# 教学实践：全国范围内的编译课程推广

## 暑期系统能力师资培训导教班







西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY　四川大学 SICHUAN UNIVERSITY　兰州大学 LANZHOU UNIVERSITY

**在教指委系统能力专家组指导下，开设两期导教班**
**学员来自西工大、川大、合工大等30所高校**
















**教学资源已被湖南大学、合工大等24所高校师生使用，覆盖校外学生5000+**

**第四届中国计算机教育大会优秀教学资源**
**2项特等奖（包揽编译方向全部奖项）**

# 本节内容提纲

1. 编译器是什么?
2. 为什么要学习编译课程?
3. 编译教学的困境与科大方案
4. AI算子编译比赛分享
5. 本学期课程设置的情况

# 国产算力算子开发大赛

为响应国家发展自主AI算力生态的战略需求，培养掌握核心技术的后备人才，组织AI国产算力算子开发创新大赛，在真实国产算力环境中，提升算子设计与调优的技术水平，创新算子开发，释放硬件潜能，发现和培养优秀人才。

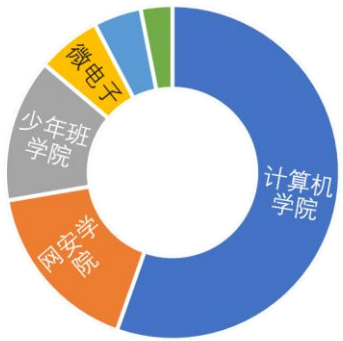
**组委会：** 中国科学技术大学 University of Science and Technology of China　HUAWEI

合肥综合性国家科学中心人工智能研究院 Institute of Artificial Intelligence，Hefei Comprehensive National Science Center

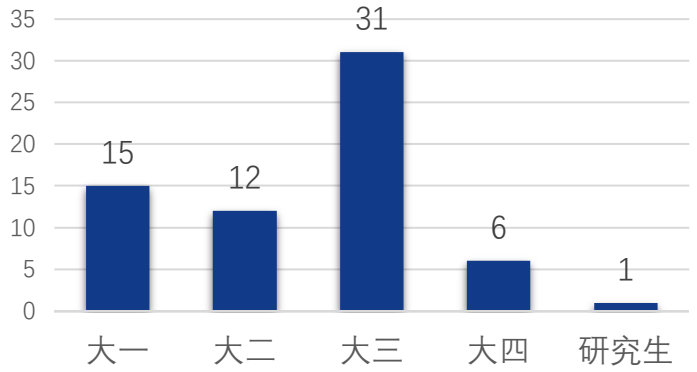
**主题：**基于Triton与TileLang先进编译技术的高性能算子开发

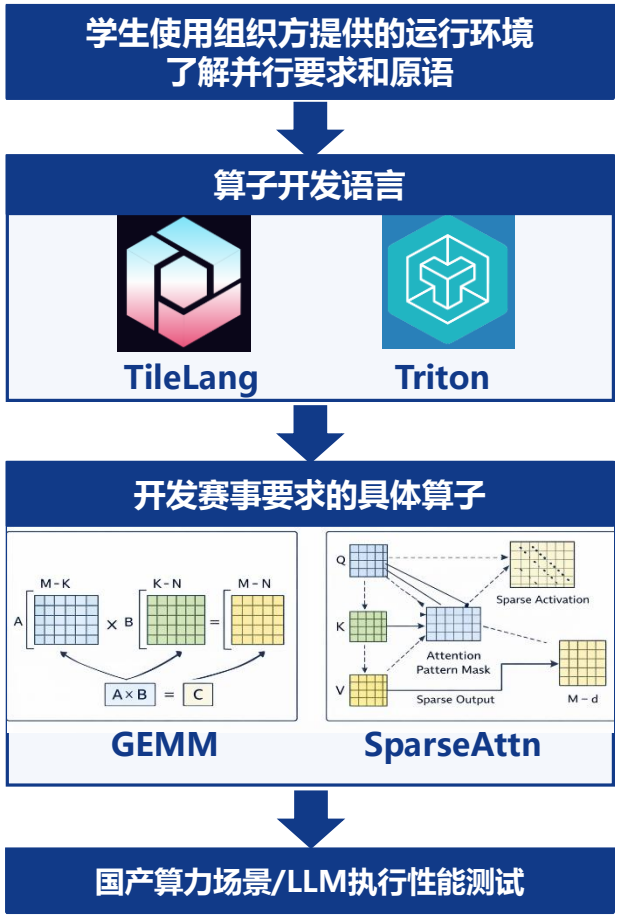
**参与人员：**全校在读本科生65名

学院分布




学生人数分布



# 国产算力算子开发大赛

## 从Softmax/MatMul基础算子到vLLM集成

**1月12日 - 1月18日**



选手报名

**1月21日 - 2月1日**



基础知识培训

并行计算 · NPU 架构 · LLM 推理 · Triton · TileLang

**2月2日 - 2月12日**



基础算子开发

Softmax · MatMul

**2月13日 - 3月3日**




Paged Attention

**3月4日 - 3月20日**



Sparse Attention

**3月23日 - 5月13日**




vLLM 集成开发与优化答辩

**能力爬坡：** 理解原理 → 编写 kernel → 性能打榜 → 系统集成

# 国产算力算子开发大赛

培训：助教团队（艾琮焜、王宇航、孙静雯、程业泰、金泽文、游兴业、肖同欢、陈子琪）

业界专家（华为2012实验室、北大杨智团队、中科院计算所崔慧敏团队）

基础认知 → 开发入口 → 算子编程 → 推理背景 → 性能诊断

## 基础知识培训

并行计算 · NPU 架构



## 开发环境搭建

统一环境 · 开箱即用



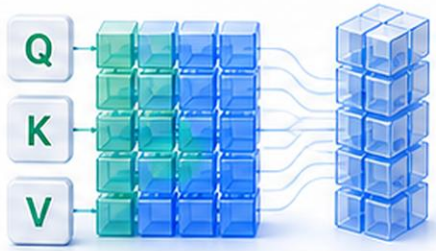
## Triton / TileLang 入门

代码更短 · 上手更快



## LLM 推理背景

Transformer · PagedAttention · Sparse Attention




## Profile 实战

msprof · MindStudio Insight

单算子 / 端到端分析



# 为什么用Triton/TileLang?

## 相比 Ascend-C，更适合本科生实践入门

### Ascend-C

比 Triton / TileLang 更贴近底层
适合生产级深度优化，但学习成本高




### Triton / TileLang

代码更短，上手更快
更适合本科生竞赛




### 真实 NPU 性能问题

仍然要面对
tile、访存、融合、并行度




**更容易入门**
代码更短，上手更快
适合作为竞赛入口

**保留真实约束**
访存带宽、并行度、
kernel launch、数据布局都绕不开

**更深的成长路径**
后续优秀选手可以继续往
Ascend-C 和更底层工具链深入

先用 Triton / TileLang 建立性能思维，保留真实 NPU 约束，再向 Ascend-C 深入。

# 评测方式

评测采用OJ方案，基于真实NPU环境评测

为充分发挥选手能力，积极**鼓励全流程AI辅助**的优化（包括调研、开发与分析）

## 机器环境

### Ascend 910B3 集群



资源来源

希冀平台 · 类脑开放平台 · 计算机学院实验教学中心

## 评测流程

按 OJ（Online Judge）典型流程进行

- 支持文件 / git 仓库提交
- 统一环境
- Docker 隔离
- PyTorch 精度对齐
- 性能打榜

# 国产算力算子开发大赛

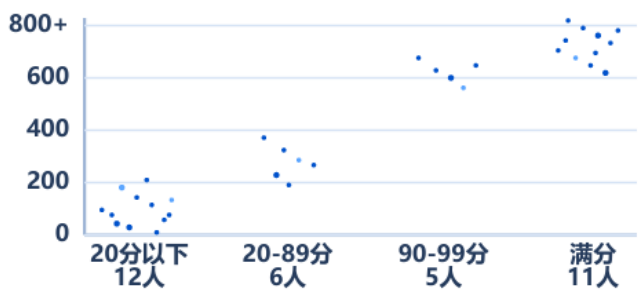
## 结果概述

本次sparse attention赛题，性能最高的**实现**在我们的测试数据规模下，与pytorch的实现相比，triton算子获得了**3.46x**的加速比。下表汇总了综合算子 paged attention 和 sparse attention 的结果：

## Paged Attention 阶段亮点

| 峰值加速比 | 加速比 800+ | 加速比 700+ |
|---|---|---|
| **850.2×** | **11人** | **16人** |
| vs baseline | 满分 | 90 分以上 |

### 成绩与加速比分布




### 重点分布

峰值加速比：850.2×

加速比 800+：11 人

加速比 700+：16 人

20 分以下：12 人（加速比 200-）

## Sparse Attention 阶段亮点

| 最高加速比 | 有效提交 | 高于 baseline |
|---|---|---|
| **3.46×** | **31人** | **20人** |
| baseline 66ms | | |

### 性能排行榜（按综合用时加速比排序）

| 排名 | 加速比 | 综合用时 | 备注 |
|---|---|---|---|
| Top 1 | 3.46× | 19.1 ms | 显著领先 |
| Top 2 | 1.51× | 43.7 ms | 高于 baseline |
| Top 3 | 1.47× | 44.9 ms | 高于 baseline |
| … | … | … | … |

* baseline = 66ms；加速比 = 66 / 综合用时，WA 不计入有效提交

# 国产算力算子开发大赛

## 完成华为vLLM-Ascend推理框架的算子集成，探索了如下的高阶优化方案

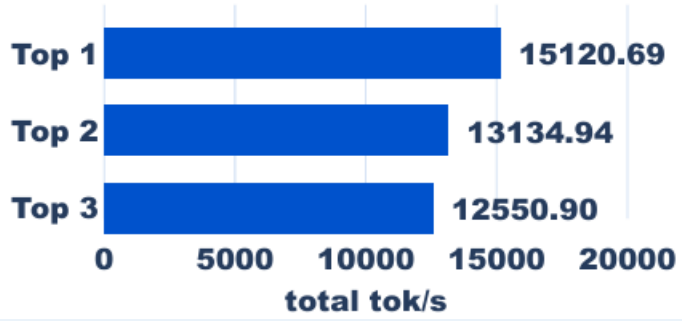
- **算子融合**：探索 top-k kernel / GQA q-head 调度等融合优化
- **算子入图**：从不同维度尝试算子入图，降低端到端调度开销
- **Pipeline / Load 优化**：结合 **MindStudio**，分析流水线与访存加载瓶颈
- **tile调参**：针对 910B 硬件特性调整分块与执行参数

## 在线Serving Top3作品性能

| Top 1 | Top 2 | Top 3 |
|---|---|---|
| Total tok/s: **12836.12** | Total tok/s: **11884.05** | Total tok/s: **11030.91** |
| Output tok/s: 282.37 | Output tok/s: 263.37 | Output tok/s: 244.46 |

**基线：Total 2696.91 tok/s, Output 194.64 tok/s, 0.31 req/s, 前三名各指标优化倍数超4.09x、1.26x、2.67x**

## 离线Serving Top3作品性能




**基线：Total 2890.33 tok/s, 前三名指标优化倍数超4.34x**

# 奖励机制

奖金按阶段即时发放，为选手的优化提供正反馈

## 赛前承诺



**基础算子开发**
Softmax + MatMul




**综合算子开发 (1)**
Paged Attention
奖金 1000 元




**综合算子开发 (2)**
Sparse Attention
奖金 +2000 元



**工业集成开发**
Integrate into vLLM

所有奖励可叠加



| | |
|---|---|
| 一等奖 | +6000 元 |
| 二等奖 | +3000 元 |
| 优胜奖 | +1500 元 |

## 最终兑现

**16.41 万元**
实际发放奖金





**案例**
**一等奖 · 吴卫昊**
实际发放 9200 元

# 本节内容提纲

1. 编译器是什么?
2. 为什么要学习编译课程?
3. 编译教学的困境与科大方案
4. AI算子编译比赛分享
5. 本学期课程设置的情况

# 课程设置

- 每周一(6,7)、三(3,4)，高新区GT-B112教室
- 课程信息化平台（2个网站1个群）：
  - 课程主页：
    - https://ustc-compiler-2026.github.io/homepage/
    - 发布讲义+实验信息
  - 希冀平台：
    - https://cscourse.ustc.edu.cn/
    - 作业提交、代码提交、自动化评测
  - qq群
    - 发布通知、非灌水群



# 参考资料

## ❑教材和参考书

- 李诚、徐伟、张昱，现代编译器设计与实现，高等教育出版社，2026
- 陈意云、张昱，编译原理（第3版），高等教育出版社，2016
- 李诚、徐伟，电子教材，https://abooks.hep.com.cn/188243
- A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman 著，赵建华等译，编译原理，机械工业出版社，2017

## ❑实验视频

- B站视频：https://www.bilibili.com/video/BV1D7421N7c4

## ❑其他资料

- Stanford课程主页
  http://web.stanford.edu/class/cs143/
- MIT课程主页：
  http://6.035.scripts.mit.edu/fa18/

# 考核要求

- 考核内容包括理论学习、工程实践
- 成绩组成：
  - 平时考核（15%）
    - 按时上课（特殊情况不能来需要书面请假）
    - 按时完成课后作业（可以豁免一次作业）
  - 考试（45-60%）
    - 期中+期末
  - 工程实践（35-40%）
  - 该比例会动态调整，最终解释权归教学团队老师所有！

# 助教天团



徐伟 博士
高级实验师



乔昱霖
博一



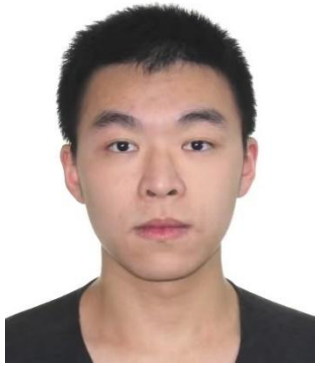
游兴业
研二



王宇航
研一



孙婧雯
研一



卿文杰
大四保研



吴卫昊
大四保研

# 本学期的实验要求（后续可能有变动）

| 阶段 | 实验名称 | 难度系数 | 时间安排 | 是否计入成绩 |
| --- | --- | --- | --- | --- |
| 基础实验 | Lab 0 系统配置 | - | 1周 | 是 |
| | Lab 1 词法语法器 | ★ | 1.5周 | 是 |
| | Lab 2 中间代码生成器 | ★★ | 2.5周 | 是 |
| 高阶实验 | Lab 3 龙芯后端代码生成器 | ★★★ | 2.5周 | 是 |
| 拔高实验 | Lab 4 代码优化 | ★★★★ | 3周 | 是 |
| 竞赛实验 | **Lab 5 AI编译** | ★★★★★★ | 不计入教学周 | 否 |

- **奖金若干+其他纪念品+强推荐信+实习推荐**

# 本课程适合什么样的学生?

- 想深入理解编译技术，并付诸实践的学生！
- 认准目标不轻易放弃的学生！
- 乐于沟通，懂得寻找方法解决问题的学生！
- 喜欢高分，但能正确认识分数与付出之间的关系的学生！
- 不是总抱怨，而是提出建设性意见的学生！
- 欢迎使用ai工具，但是不要过度依赖！

# 慎重，可退课，if, but not limited to




- 不想花太多时间在课程实践上
- 不喜欢合作，不愿意交流，不懂得沟通
- 不喜欢提前规划学习，总是临时抱佛脚
- 心理承受能力比较低
- 做事情虎头蛇尾
- 希望考高分，但不能正确理解分数与付出间的正确关系
- 严重依赖往届开源实现，寄希望于CSDN、chatGPT、DeepSeek

请参考评课社区分数0分或者1分的评论！

# 第一次作业

- 用ai生成PPT
- 主题为“ai时代是否还要学习coding和编译？”
  - 先选择立场，是与否
- 总评加分不超过0.5分
  - 形式+内容+ai工具使用过程说明
- ddl：9月14日
- 提交方式：
  - ppt及其材料（if有一些动画或者视频等）
  - 工具使用、PPT交互式修改过程报告（形式和体裁不限）




# 教会AI写代码，但由你定义灵魂！

李 诚

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2026年08月31日